# An Implementation of PPO

Kieran Paranjpe

July 2026

## 1 Abstract

The goal of this project is to deepen my understanding of deep reinforcement learning by implementing Proximal Policy Optimisation (PPO). This report describes the theory, implementation details of the project and results. I will also include areas which were difficult to work out, how I troubleshot, and what I learned. I trained policies on 5 tasks from Gymnasium [1], each one achieving successful results.

**Click for source code**
**Click for Weights & Biases dashboard**

## 2 Introduction

Before this project, I attempted different reinforcement learning projects where I implemented algorithms myself. The first was an implementation of tabular SARSA applied to a gridworld. This produced satisfactory results and was effective as a simple first reinforcement learning project. Following this project, I attempted to train a humanoid in MujoCo to walk by using REINFORCE with eligibility traces, as explained in the second edition of Sutton and Barto [2]. Unfortunately, this project was too ambitious, and did not produce successful results.

As a result, I decided to start this project with two key differences from my previous attempt. The first and most obvious difference is the choice of algorithm; PPO is considered the go-to algorithm for these sorts of problems. The second difference was my approach to the framework. It was important to me that I could easily reuse the same training loop and algorithm on many different tasks of varying difficulty, so I could debug more easily. If the task itself is hard to learn, it's hard to know if the algorithm is wrong, if the hyperparameters are wrong, or if I just need to train for longer. Starting with simple tasks solves this problem.

Although this report is written following the format of a paper, it is not. It's almost a report, but mostly an exercise for myself to write down what I did to solidify my understanding.

## 3 Background

### 3.1 The Markov Decision Process (MDP)

The reinforcement learning problem is the problem of finding the optimal **policy** to solve a given **Markov Decision Process** (MDP). An MDP can be viewed as a 4-tuple:

$$M = (S, A, \mathcal{P}(s'|a, s), \mathcal{R}(a, s, s'))$$

where $S$ is the set of states and $A$ is the set of actions [2, 3]. $\mathcal{P}(s'|a, s)$ is the probability of ending up in state $s'$ given that we are in state $s$, and take action $a$. $\mathcal{R}(a, s, s') : (A \times S \times S) \to \mathbb{R}$ is the reward function that gives the reward for being in state $s$, taking action $a$, and ending up in state $s'$. We often use the shorthand $r_t = \mathcal{R}(a_t, s_t, s_{t+1})$ to denote the reward at timestep $t$. The policy is defined as the distribution over actions given the state [2], parameterised by weights $\theta$:

$$\pi_\theta(a|s)$$

When we "run" the environment, we collect a sequence of states, actions and rewards: $s_1, a_1, r_1, ..., s_T, a_T, r_T, s_F$, for $T$ timesteps, where $s_F$ is the final state. Each timestep, we sample $a_t \sim \pi_\theta(\cdot|s_t)$ to (hopefully) pick the action that maximises the total reward.

A note on notation: I will use $\mathcal{P}(x|y)$ to denote the probability of $x$ given $y$, and $\mathcal{P}(\cdot|y)$ to denote the entire distribution from which $x$ is sampled.

## 3.2 Value and Action-Value Functions

Before I explain my implementation of PPO, it is crucial to understand value functions and action-value functions. While this is not specific theory to PPO, it is integral to the foundation behind actor-critic methods; the value function is the critic.

See my dedicated notes on value functions and action-value functions for more details. In summary:

### 3.2.1 Value Function

The value function quantifies how good it is to begin in a given state $s$, and then follow and sample actions from a policy $\pi$ until the episode ends. It is defined as:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[\sum_{k=t}^{T} \gamma^{k-t} r_k \mid S_t = s] = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

where $G_t$ is called the return, and $T$ is the length of the episode (in principle this can be $\infty$) [2, 3]. The value function is the expected return when following the policy $\pi$, starting with $S_t = s$. We can empirically compute one specific value for $G_t$ by doing a full episode rollout and computing the sum of rewards. The value $\gamma$, called the discount factor, is a hyperparameter that allows us to weigh earlier rewards more than later rewards.

### 3.2.2 Action-Value Function

The action-value function is defined as:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

where $a$ can be *any* action, not just an action sampled from $\pi$ [2, 3]. This is very similar to the value function, but instead of only representing the expected return when following policy $\pi$ and starting with $S_t = s$, we also condition on starting with taking action $A_t = a$. This means that the $Q$ function tells us how good it is to be in state $s$, immediately take action $a$, and then continue to take actions according to the policy $\pi$.

### 3.2.3 $V$ in terms of $Q$

The only difference between $V$ and $Q$ is how the first action is chosen. In the case of the value function, we always sample the action from the policy at every timestep. For the action-value function we sample the action from the policy at every timestep *except* the first action. As such, we can write the value function as:

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^{\pi}(s, a)] = \sum_{a \in A} \pi(a|s) Q^{\pi}(s, a)$$

Because the expectation of the $Q$ function is the value function, it is an unbiased estimator of the value function. This can be very helpful in estimating the value function.

### 3.2.4 Advantage function

Because the $Q$ function does not force us to only consider actions sampled from $\pi$, it can be quite useful in determining if an action which would not be sampled in our current policy is better or worse than the action that would be sampled from our current policy. We can quantify this with the advantage function, defined as follows:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

This tells us how much better it is to take action $a$, and then follow $\pi$, than it is to follow only actions sampled from $\pi$.

# 4 Methods

## 4.1 PPO

PPO consists of two phases: rollout/collection and policy/value function updates [4, 5]. During the first phase, we step through time and sample actions from the policy, $\pi_\theta(a|s)$, and during the second phase, we learn from the previously gathered experience to improve the policy. The two phases alternate and repeat, so during one run of PPO, we will see phase 1, phase 2, phase 1, phase 2, etc.

### 4.1.1 Phase 1: Rollout/Collection

During phase 1, we run the agent in the environment to collect its experience. We repeat the following steps for each timestep $t$:

1. Observe state $s_t$
2. Sample action $a_t \sim \pi_\theta(\cdot|s_t)$ and take action $a_t$
3. End up in $a_{t+1} \sim \mathcal{P}(\cdot|a_t, s_t)$
4. Get reward $r_t = \mathcal{R}(a_t, s_t, s_{t+1})$ from the environment
5. Repeat from (1) and increment $t$.

This sequence represents the behaviour of a single timestep. For each timestep taken, we record the information into a replay buffer so we can train on it in phase 2. The replay buffer is a list of tuples:

$$\mathbb{D}_t = (s_t, a_t, r_t, s_{t+1}, \ln \pi_\theta(a_t|s_t), \tau_t)$$

where $s_t$, $a_t$, and $r_t$ are the state, action and reward at the current timestep, $s_{t+1}$ is the next state, $\ln \pi_\theta(a_t|s_t)$ is the log probability of taking the action $a_t$, and $\tau_t \in \{\text{IN PROGRESS}, \text{TERMINATED}, \text{TRUNCATED}\}$ tells whether the episode is done or in progress at this timestep. $\tau_t = \text{TERMINATED}$ means that the episode organically ended (like a humanoid falling over), and $\tau_t = \text{TRUNCATED}$ means that the episode reached its maximum length. Note that we actually store each state twice, once as the next state at timestep $t$, and also as the current state in timestep $t+1$. It's not necessary to do this, but it makes the implementation easier. We continue to step through time and build the replay buffer until it reaches its capacity, a configurable hyperparameter, and then we end phase 1 and move on to phase 2.

### 4.1.2 Policy / Value Function Updates

During phase 2, we compute the gradients of the respective loss functions for the policy and value functions, and update their weights according to the gradients. We can think of this phase as a supervised learning problem, where the dataset is the replay buffer. We do $k$ updates to the policy and value function with the same dataset, where $k$ is configurable.

#### Policy Updates

Since we keep track of two different neural networks, one for the policy and one for the value function, we have 2 different loss functions for each. The loss function for the policy is given by:

$$L_{\pi_\theta}(\theta) = -\frac{1}{|\mathbb{B}|}\sum_{t\in\mathbb{B}}[\min(R_t(\theta)\cdot\hat{A}_t^{\text{std}}, \text{clip}(R_t(\theta), 1-\epsilon, 1+\epsilon)\cdot\hat{A}_t^{\text{std}}) + \beta S^{\pi_\theta}(s_t)]$$

where $\mathbb{B}$ is a mini-batch sampled from the replay buffer, $\beta$ is a configurable hyperparameter, $S^{\pi_\theta}(s_t)$ is the entropy of the policy, the probability ratio $R_t(\theta)$ is

$$R_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

and the advantage $\hat{A}_t$ is computed with generalised advantage estimation (GAE) [6], then standardised (see later):

$$\hat{A}_t = \sum_{k=t}^{T}(\gamma\lambda)^{k-t}\delta_k$$

with TD error given by:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

In practice, we compute $R_t(\theta)$ using logarithmic probabilities and magnitude clipping for numerical stability.

$$R_t(\theta) = \text{clip}(e^{l_1 - l_2}, -c_1, c_1)$$

where

$$l_1 = \text{clip}(\ln \pi_\theta(a_t|s_t), -c_2, c_2)$$

and

$$l_2 = \text{clip}(\ln\ \pi_{\theta_{\text{old}}}(a_t|s_t), -c_2, c_2)$$

For my implementation, I hard-coded $c_1 = 10$ and $c_2 = 40$.

Note that we compute action probabilities with respect to 2 different distributions: $\pi_\theta(a_t|s_t)$ and $\pi_{\theta_{\text{old}}}(a_t|s_t)$. $\theta$ represents the weights vector of our current policy, and changes with each update. $\theta_{\text{old}}$ is the weights of the policy that was used to complete the most recent phase 1. We do not need to store these weights in memory, as we store the log probability in the replay buffer.

Similarly to the probability ratio, we also compute the advantage slightly differently in practice. After phase 1, and before the beginning of phase 2, we compute the advantage $\hat{A}_t$ for each timestep in the replay buffer. We do so recursively, using the recurrence relation derived as follows:

$$\begin{aligned}
\hat{A}_t &= \sum_{k=t}^{T} (\gamma\lambda)^{k-t}\delta_k && \text{Starting definition} \\
&= (\gamma\lambda)^0\delta_{t+0} + \sum_{k=t+1}^{T} (\gamma\lambda)^{k-t}\delta_k && \text{Pull out the first term of the sum} \\
&= \delta_t + (\gamma\lambda) \sum_{k=t+1}^{T} (\gamma\lambda)^{k-t-1}\delta_k && \text{Factor out } \gamma\lambda \\
\hat{A}_t &= \delta_t + (\gamma\lambda)\hat{A}_{t+1} && \text{Substitute } \hat{A}_{t+1}.
\end{aligned}$$

Finally, we standardize all the advantages, and use $\hat{A}_{\text{std}}$ in loss calculations:

$$\hat{A}^{\text{std}} = \frac{\hat{A} - \mathbb{E}(\hat{A})}{\text{Var}(\hat{A})}$$

Using this relation, we can compute all advantages in the replay buffer in $O(|\mathbb{D}|)$ instead of $O(|\mathbb{D}|^2)$.

**Value function updates**

The loss function for the value function is given by the L2 loss between $V_\Theta(s_t)$ and $\hat{A}_t + V_{\Theta_{\text{old}}}(s_t)$, where $\Theta$ is the weights of the value function.

$$L_{V_\Theta(s_t)} = \frac{1}{|\mathbb{B}|} \sum_{t \in \mathbb{B}} [V_\Theta(s_t) - (\hat{A}_t + V_{\Theta_{\text{old}}}(s_t))]^2$$

Why do we use $\hat{A}_t + V_{\Theta_{\text{old}}}(s_t)$ as our value function target? As I mentioned in the background section, $Q$ is an unbiased estimator of $V$, so any estimation of $Q$ can work as a target value function. Instead of keeping track of an independent estimation of $Q$, we notice that

$$Q^\pi(s, a) = A(s, a) + V^\pi(s)$$

So we can just use our running estimation of the advantage and value function to estimate $Q$, which in turn is an estimation of the value function.

### 4.2 Policy Network

In the last section, I referred to the policy as a neural network, but I did not explain its architecture nor how to sample from it. This section will remedy this. The policy is a distribution $\pi(\cdot|s)$ over actions given the state. The optimal policy should assign maximum probability to the action which maximises the return $G_t$. It is tempting to define the policy not as a distribution, but instead as a function $\pi : S \to A$. We do not do this, because then it would prevent us from introducing randomness into our policy. There are scenarios where the optimal action cannot be chosen deterministically. Additionally, introducing some randomness allows the policy to try actions that it hasn't seen yet, so it can learn from more diverse experience.

We create a distribution from a neural network by parameterising a distribution based on the outputs of the network [5]. I implemented this for two types of distributions: Categorical for discrete action spaces, and Beta for continuous action spaces.

#### 4.2.1 Categorical Policy

We define the categorical policy as:

$$\pi_\theta(a|s) = \text{Categorical}(f_\theta(s))$$

where

$$Y = f_\theta(s),\ Y \in \mathbb{R}^{|A|}$$

is a neural network with weights $\theta$ mapping from states to action probabilities. This is essentially just a classification problem.

#### 4.2.2 Beta Policy

In this case, the action space is continuous, and we have $|A|$ continuous actions thus $a \in \mathbb{R}^{|A|}$. This means we need to sample from $|A|$ independent beta distributions to sample one action. We define the policy as:

$$\pi_\theta(\cdot|s) = \begin{bmatrix} \text{Beta}(\alpha_1, \beta_1) \\ \text{Beta}(\alpha_2, \beta_2) \\ ... \\ \text{Beta}(\alpha_{|A|}, \beta_{|A|}) \end{bmatrix},\ \pi_\theta(\cdot|s)_i = \text{Beta}(\alpha_i, \beta_i) \forall i \in [1, |A|]$$

where

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \text{softplus}[f_\theta(s)] + 1.02\ :\ \alpha, \beta \in \mathbb{R}^{|A|}$$

then the probability of taking an action is:

$$\pi_\theta(a|s) = \prod_{i=1}^{|A|} \pi_\theta(a_i|s)_i$$

and we can sample each action like so:

$$a_i \sim \pi_\theta(\cdot|s)_i$$

The output of the neural network is $f_\theta(s) \in \mathbb{R}^{2|A|}$. We use softplus and add 1.02 to ensure the output is always positive and greater than 1.02. This is because the beta distribution is not helpful to us when $\alpha, \beta < 1$. The benefit of using the beta distribution is that each action $a_i \sim \pi_\theta(\cdot|s)_i$ is bounded between 0 and 1. It is common for tasks in control to have continuous actions with a bounded domain. However, we don't want to limit the expressivity of the beta policy to only actions bound from 0 to 1. Ideally, we want to output actions bound over in interval. As such, for a sampled action $u \sim \pi_\theta(\cdot|s)$, the action we actually use is:

$$a = g(u) = pu + q$$

where $p$ and $q$ are configurable real numbers.

For $f_\theta(s)$, I used a multi-layer perceptron with two hidden layers of size 64, with tanh activation function.

### 4.2.3 Computing the Log Probability

During the PPO update, we use the quantity $\ln \pi(a|s)$, the natural logarithm of the probability of choosing the action. In the categorical case this is trivial and is literally just the log of the probability of choosing the action. In the multi-dimensional-transformed beta case, this is not so simple.

Let $a = pu + q$ be the action and $u \sim \pi_\theta(\cdot|s)$. Then

$$\ln[\pi_\theta(a|s)] = \ln[\pi_\theta(u|s)|\det(\frac{\partial u}{\partial a})|] \quad \text{By laws of transformations of random variables multiply by the determinant of the jacobian.}$$

$$= \ln[\prod_{i=1}^{|A|} \pi_\theta(u_i|s)_i] + \ln[|\det(\frac{\partial u}{\partial a})|] \quad \text{Split single probability into product of independent probabilities.}$$

$$= \sum_{i=1}^{|A|} [\ln \pi_\theta(u_i|s)_i] + \ln[\frac{1}{p^{|A|}}] \quad \frac{\partial u}{\partial a} \text{ is an } |A| \times |A| \text{ diagonal matrix, values } 1/p \text{ on main diagonal. } \det(\frac{\partial u}{\partial a}) \text{ is the product along the main diagonal.}$$

$$= \sum_{i=1}^{|A|} [\ln \pi_\theta(u_i|s)_i] - |A| \ln[p] \quad \text{Final log probability.}$$

# 5 Results

I ran this implementation of PPO on 5 different Gymnasium tasks. The link to the Weights & Biases logs contains the results, sample videos, and hyperparameters used for each run.

1. Cart Pole
2. Lunar Lander
3. Half Cheetah
4. Walker 2D
5. Humanoid

# 6 Conclusion, Troubles and Lessons

I first had the idea for a project like this last August, when I first started watching Google DeepMind × UCL lectures on the topic. It's nice to see that almost a full year later I've achieved successful results.

One of the key differences in this project versus other attempts I've had in the past was using Gymnasium tasks and incrementally increasing the difficulty of the task. In the past, I tried to train a humanoid walking task immediately. In addition, I tried to construct my own environment and MDP. This took effort away from what really mattered: the algorithm implementation.

The version of the algorithm I described in this report was not the first thing I tried. I had many issues along the way:

1. Continuous action space bounded to $[-1, 1]$ for some tasks and $[-0.4, 0.4]$ for other tasks. Gymnasium humanoid tasks are different from other tasks, and each action has a maximum magnitude of 0.4. This forced me to generalise the translation I made after sampling the beta policy.
2. Correctly computing the log probability. I did not account for the transformation of random variables initially.
3. Standardising the advantage improved results.
4. Standardising the reward and observations from the MDP also improved results (not described in methods because it is supported by Gymnasium by default).
5. I had observed many instances of nan and $\infty$. Clipping values helped to solve this.

All in all, it was cool to see the results and I learned a lot. Thanks for reading!

## References


[1] M. Towers *et al.*, "Gymnasium: A Standard Interface for Reinforcement Learning Environments," Farama Foundation, 2023. [Online]. Available: https://github.com/Farama-Foundation/Gymnasium. [Accessed: Jun-2026].

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf. [Accessed: Jun-2026].

[3] D. Silver, "Introduction to Reinforcement Learning," University College London and DeepMind, 2015. [Online]. Available: https://davidstarsilver.wordpress.com/teaching/. [Accessed: Jun-2026].

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347. [Accessed: Jun-2026].

[5] C. Finn, "CS224R: Deep Reinforcement Learning," Stanford University, Spring 2025. [Online]. Available: https://cs224r.stanford.edu/. [Accessed: Jun-2026].

[6] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1506.02438. [Accessed: Jun-2026].